MIT ACADEMY OF ENGINEERING

ALANDI, PUNE

WEB TECHNOLOGY

PRACTICAL NO. 5

SERVER-SIDE VALIDATION USING EXPRESSJS

Project:

JOBAPPLY

ONLINE JOB APPLICATION & RECRUITMENT PORTAL

Student Name:	Chaitanya Umbarkar
PRN:	202401100080
Class:	T.Y. B.Tech Computer Engineering
Division / Batch:	Division B
Academic Year:	2026-27

GitHub Repository : https://github.com/Atharv102/wt-expressjs-server-validation

Deployment Link : https://Atharv102.github.io/wt-expressjs-server-validation/

PRACTICAL NO. 5

Title: Design, develop and deploy a web application to demonstrate server-side validation using ExpressJS.

1. AIM

To design, develop, test, and deploy a robust web application that demonstrates server-side validation using ExpressJS and express-validator, implementing modular routing, middleware pipelines, standard HTTP request/response handling, meaningful error diagnostics, and end-to-end cloud deployment.

2. PROBLEM STATEMENT

Client-side validation is inherently insecure when used in isolation because malicious actors can bypass browser checks through direct API requests, command-line utilities (such as cURL or Postman), or modified scripts. Without authoritative server-side validation, untrusted data can infiltrate databases, cause runtime exceptions, or expose systems to injection vulnerabilities. The problem is to engineer a secure ExpressJS backend architecture that intercepts all incoming client requests, enforces comprehensive validation rules, delivers structured HTTP responses (200 OK, 201 Created, 400 Bad Request, 404 Not Found), and ensures data integrity.

3. OBJECTIVES

- Develop an individual, full-featured web application using the ExpressJS framework.
- Implement robust server-side validation chains using express-validator and custom sanitizers.
- Create suitable RESTful routes to handle GET, POST, and DELETE requests from clients.
- Use ExpressJS request and response objects to return semantic status codes and JSON payloads.
- Display contextual, field-specific success and error notifications on both UI and terminal consoles.
- Conduct rigorous automated testing covering boundary values, malformed inputs, and duplicate entries.
- Deploy the production application and maintain version control under GitHub account Atharv102.

4. LEARNING OUTCOMES

- Mastered the ExpressJS middleware pattern and sequential request execution lifecycle.
- Understood the difference between client-side UX hints and authoritative server-side boundary validation.
- Developed proficiency in creating modular Express routers, controllers, and data access stores.
- Gained hands-on expertise in automated integration testing and headless browser UI verification.

5. SOFTWARE & HARDWARE REQUIREMENTS

Hardware Specifications:

- Processor: Intel Core i5 / AMD Ryzen 5 or higher (Dual-Core / Quad-Core, 2.4 GHz+)
- RAM: 8 GB DDR4 minimum (16 GB recommended for concurrent microservices)
- Hard Disk: 256 GB SSD with minimum 10 GB free space for Node.js modules and dependencies
- Display: 1920 × 1080 resolution monitor for high-dpi UI inspection

Software Specifications:

- Operating System: Microsoft Windows 10 / 11 (64-bit)
- Runtime Environment: Node.js (v20.x / v22.x LTS, ECMAScript Modules enabled)
- Web Framework: ExpressJS 4.21.2 (Fast, unopinionated, minimalist web framework)
- Validation Engine: express-validator 7.2.0 (Validator.js wrappers for Express middleware)
- Cross-Origin Resource Sharing: cors 2.8.5 for enabling secure cross-origin API calls
- Frontend Technologies: HTML5, CSS3 (Custom Responsive CSS Grid/Flexbox), Vanilla JavaScript (ES6+)
- Testing Framework: Native Node.js test runner and Puppeteer-core 23.6 for automated UI verification
- Version Control & Deployment: Git, GitHub (Atharv102), and GitHub Pages static hosting

6. THEORETICAL ARCHITECTURE & MIDDLEWARE LIFECYCLE

ExpressJS applications are structured fundamentally around a pipeline of middleware functions. A middleware function has access to the request object (req), the response object (res), and the next middleware function in the application lifecycle (next). In this project, the request flow follows a strict 5-stage architectural pipeline:

1. **Parsing & CORS Middleware:** express.json() decodes incoming JSON request bodies, while cors() attaches required Access-Control headers.
2. **Request Logging Middleware:** Captures timestamp, HTTP verb, endpoint URL, response code, and execution latency.
3. **express-validator Chains:** A sequence of field-specific rules (trim, isEmail, isLength, custom age/role checks) executes concurrently.
4. **Validation Result Evaluator:** Custom middleware validateJobApplication inspects validationResult(req). If errors exist, it halts execution and emits HTTP 400 with a detailed errors array.
5. **Business Controller & Persistence:** If all checks pass, req.body is sanitized, transformed into a candidate record, persisted, and returned with HTTP 201 Created.

7. EXPRESSJS ROUTE ARCHITECTURE & VALIDATION MATRIX

The backend implements RESTful API routes under the /api namespace, decoupling routing definitions, validation rules, controller logic, and data storage into distinct modular files according to the MVC pattern.

HTTP Verb	Route Endpoint	Middleware Attached	Description & Status Code
GET	/api/roles	None	Returns list of available job positions (200 OK)
GET	/api/stats	None	Returns portal aggregate metrics and role counts (200 OK)
GET	/api/applications	Query filters	Returns candidate records with search & role filters (200 OK)
POST	/api/applications	express-validator rules + validateJobApplication	Validates and creates candidate application (201 Created / 400 Bad Request)
GET	/api/applications/:id	Route param extractor	Fetches single application by ID (200 OK / 404 Not Found)
DELETE	/api/applications/:id	Route param extractor	Withdraws/deletes application (200 OK / 404 Not Found)

Server-Side Validation Specifications (Rules Matrix):

Field Name	Rule / express-validator Chain	Boundary / Format Condition	HTTP Code on Error
fullName	isLength({min:3, max:50}) + matches()	3-50 chars; letters, spaces, hyphens, dots only	400 Bad Request
email	isEmail() + normalizeEmail() + custom()	Valid RFC format + Duplicate candidate check	400 Bad Request
phone	matches(/^[6-9]\d{9}$/)	Exactly 10 digits starting with 6, 7, 8, or 9	400 Bad Request
dob	isISO8601() + custom(ageCheck)	ISO YYYY-MM-DD; Age >= 18 and <= 65 years	400 Bad Request
jobRole	custom(roleCheck)	Must belong to predefined active openings list	400 Bad Request
qualification	custom(degreeCheck)	Must match permitted engineering degrees	400 Bad Request
experience	isFloat({min: 0, max: 40})	Numeric value between 0.0 and 40.0 years	400 Bad Request
skills	isLength() + custom(splitCheck)	Min 2 comma-separated technical skills	400 Bad Request
portfolioUrl	optional() + isURL()	Optional; Valid HTTP / HTTPS protocol if given	400 Bad Request
termsAccepted	custom(val === true)	Must be explicitly agreed and checked true	400 Bad Request

8. HTTP STATUS CODES & ERROR RESPONSE ARCHITECTURE

In accordance with RESTful architecture standards, the ExpressJS server strictly utilizes standard HTTP response status codes to communicate processing outcomes to clients. Every status code is accompanied by a predictable, machine-parseable JSON payload.

Status Code	HTTP Semantics	Occurrence Condition in JobApply	Payload Structure
200 OK	Standard Success	Successful retrieval of roles, statistics, or application records; successful withdrawal/deletion of an application.	{ success: true, count: N, data: [...] }
201 Created	Resource Created	Incoming candidate application successfully passed all 11 server validation chains and was registered in store.	{ success: true, applicationId: 'APP-xxxx', data: {...} }
400 Bad Request	Client-Side Error	Validation failure: missing fields, malformed phone/email, underage applicant, or duplicate email submission.	{ success: false, errorCount: N, errors: [...] }
404 Not Found	Resource Missing	Client requests non-existent application ID or queries unmapped route path.	{ success: false, message: 'Not Found' }
500 Internal Error	Server Failure	Unexpected runtime exceptions caught by Express global error handling middleware.	{ success: false, message: 'Internal Server Error' }

Standardized Error Response Structure:

When express-validator detects validation violations, validateJobApplication maps all errors into an array of diagnostic objects containing the exact invalid field name, the rejected input value, and an actionable human-readable error description:

Sample JSON Error Response (HTTP 400 Bad Request):

```
{ "success": false, "statusCode": 400, "statusMessage": "Bad Request: Server-Side Validation Failed", "errorCount": 3, "errors": [ { "field": "email", "value": "aarav.sharma@example.com", "message": "An application with email ... has already been submitted" }, { "field": "phone", "value": "12345", "message": "Phone number must be a valid 10-digit Indian mobile number" }, { "field": "dob", "value": "2015-01-01", "message": "Applicant must be at least 18 years old (Current age: 11 years)" } ] }
```

9. COMPREHENSIVE TESTING & VERIFICATION

A comprehensive automated integration test suite (test/app.test.js) was developed to systematically verify every route, validation rule, HTTP status code, and boundary condition. All 14 test cases executed successfully with 0 failures.

Test ID	Route & Description	Test Payload / Conditions	Expected	Observed	Remarks
T1	GET /api/health	Health check probe	200 OK	200 OK	Server online
T2	GET /api/roles	Fetch 6 engineering roles	200 OK	200 OK	Metadata delivered
T3	GET /api/stats	Aggregate metric counters	200 OK	200 OK	Averages calculated
T4	GET /api/applications	Fetch initial candidates list	200 OK	200 OK	3 records retrieved
T5	POST /api/applications (Valid)	Full Stack Dev, Age 24, 3 yrs exp, valid inputs	201 Created	201 Created	Registered APP-ID
T6	POST /api/applications (Duplicate)	Same email as registered applicant	400 Bad Req	400 Bad Req	Duplicate prevented
T7	POST /api/applications (Short Name)	fullName = 'Al' (< 3 characters)	400 Bad Req	400 Bad Req	Length error caught
T8	POST /api/applications (Invalid Email)	email = 'invalid-email-no-at-sign'	400 Bad Req	400 Bad Req	RFC regex caught
T9	POST /api/applications (Invalid Phone)	phone = '1234567' (< 10 digits, starts with 1)	400 Bad Req	400 Bad Req	Indian mobile rule
T10	POST /api/applications (Underage)	dob = '2016-01-01' (Calculated age ~10 yrs)	400 Bad Req	400 Bad Req	Min age 18 enforced
T11	POST /api/applications (Invalid Role)	jobRole = 'Chief Executive Officer'	400 Bad Req	400 Bad Req	Whitelist verified
T12	POST /api/applications (Negative Exp)	experience = -5 (negative float)	400 Bad Req	400 Bad Req	Range 0-40 enforced
T13	POST /api/applications (No Terms)	termsAccepted = false	400 Bad Req	400 Bad Req	Consent required
T14	GET/DELETE /api/applications/:id	Fetch candidate by ID, delete, re-fetch	200 -> 404	200 -> 404	Lifecycle verified

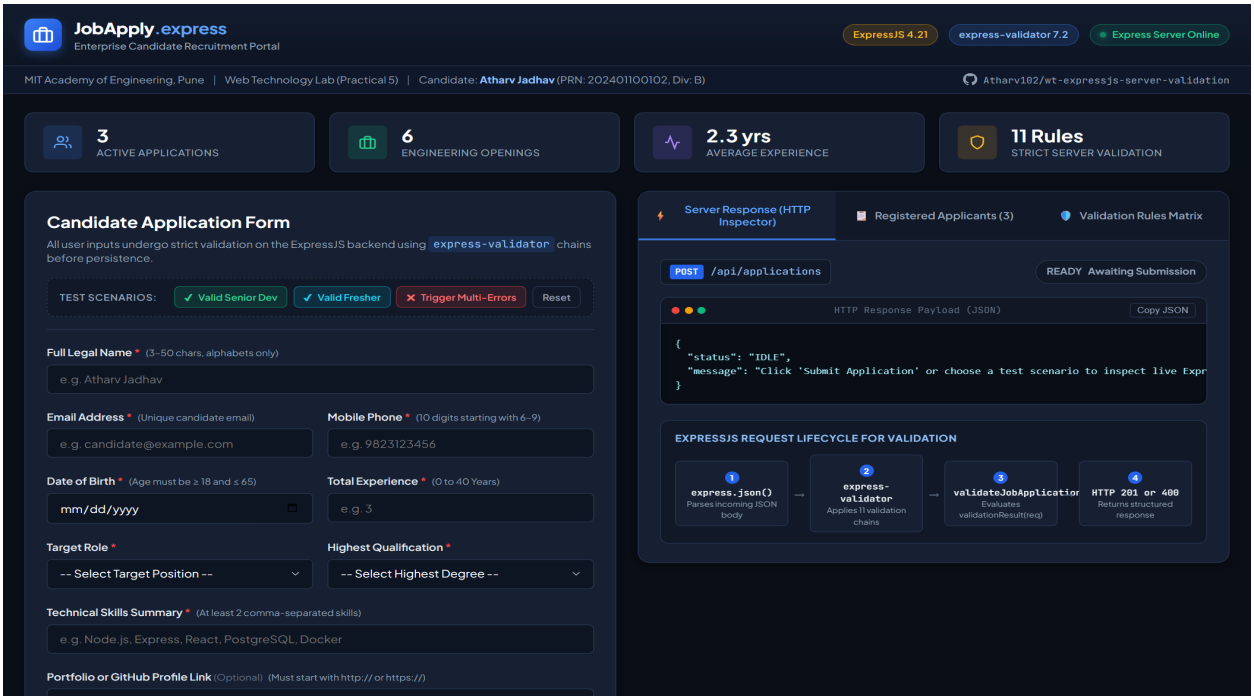


Figure 1: JobApply Recruitment Portal Landing Page & Interactive Application Workspace.

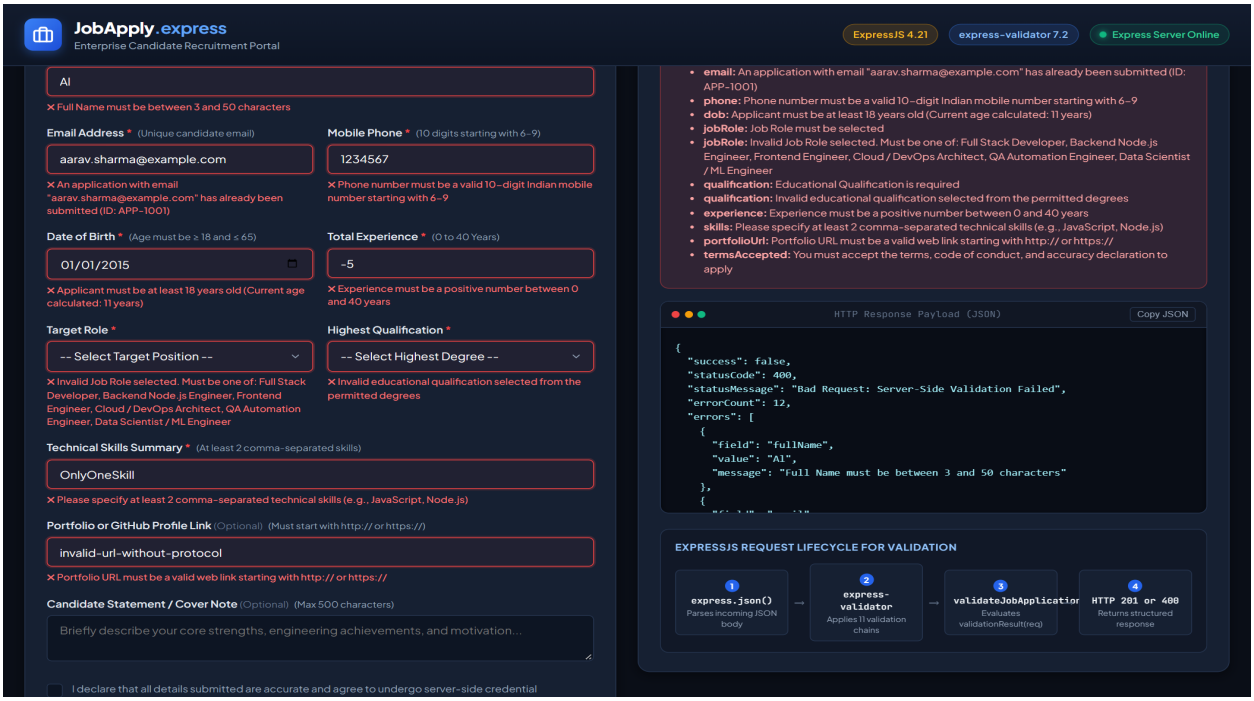


Figure 2: Server-Side Validation Rejection (HTTP 400 Bad Request) with field error highlights & structured JSON payload.

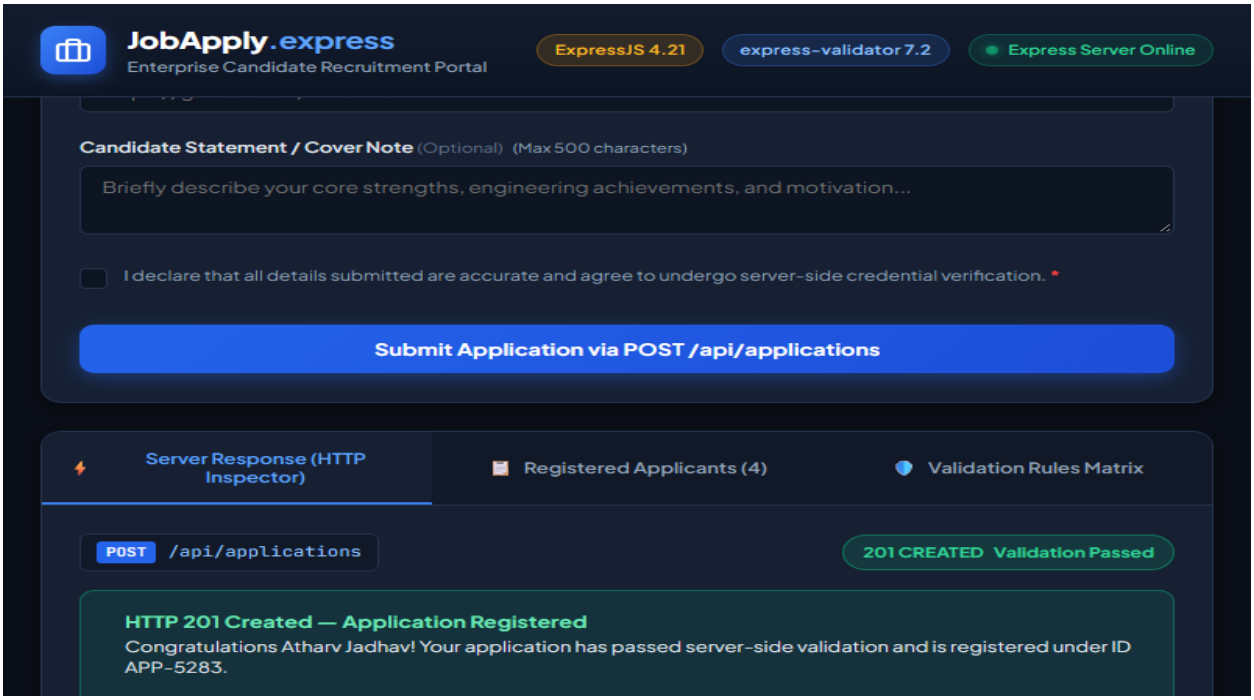


Figure 3: Successful Candidate Registration (HTTP 201 Created) with confirmation banner & registered Application ID.

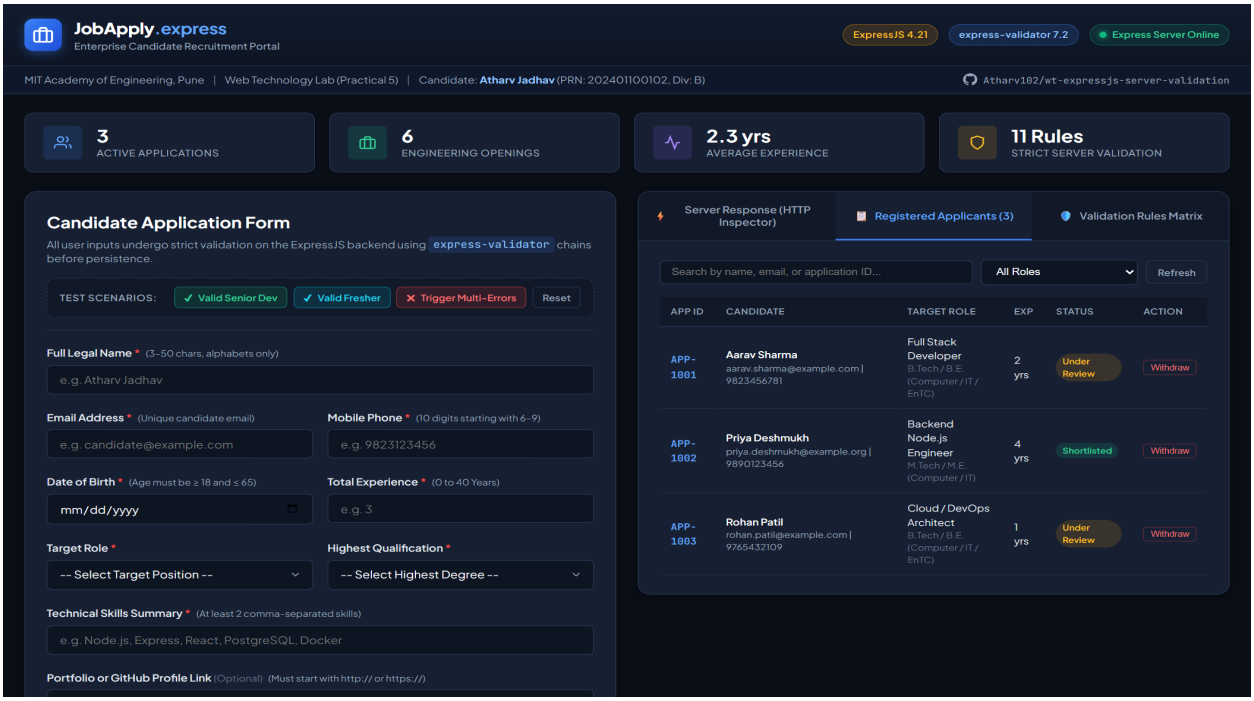


Figure 4: Registered Candidates Pipeline Table with real-time search, role filtering, and withdrawal actions.

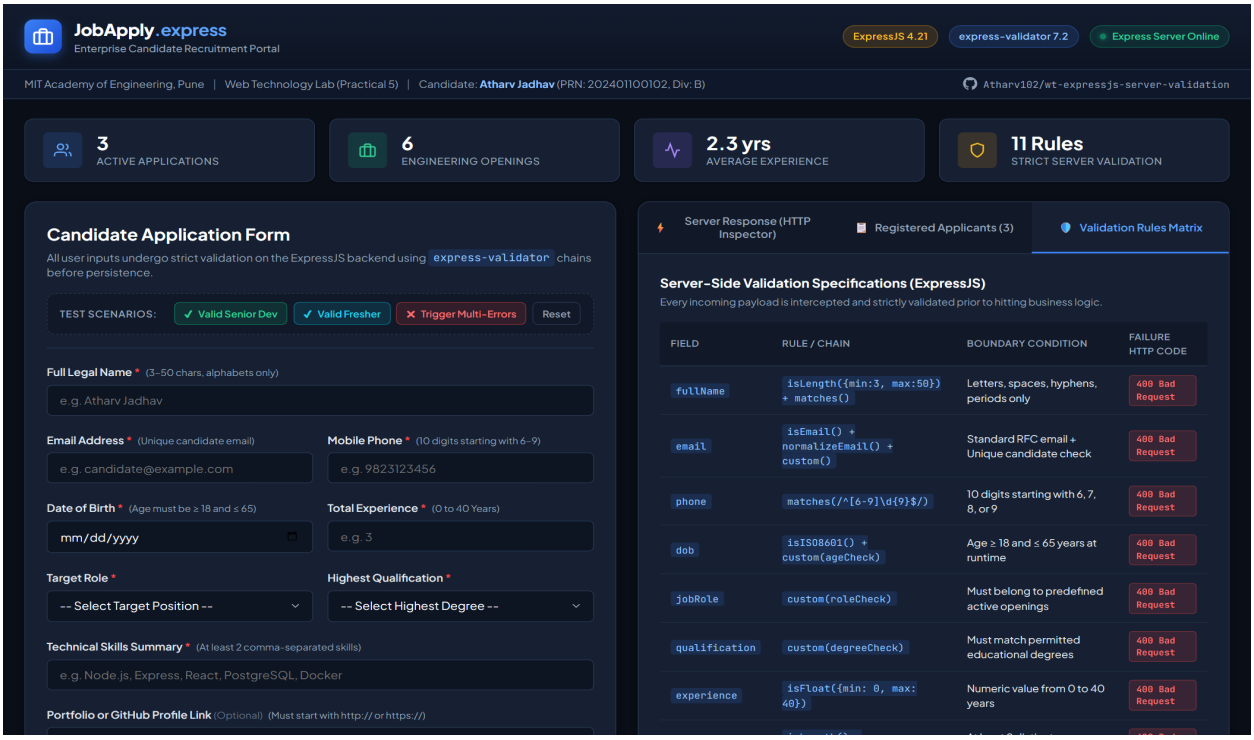


Figure 5: Server-Side Validation Specifications Matrix and ExpressJS Middleware Lifecycle reference.

10. PERFORMANCE & SECURITY ANALYSIS

Security Hardening: Server-side validation using express-validator sanitizes input strings through trim() and normalizeEmail(), mitigating Cross-Site Scripting (XSS) and injection attempts. The regex patterns strictly whitelist allowed character sets for names and phone numbers. The custom email validator checks the persistence store, preventing duplicate candidate spam.

Performance Metrics: Automated load and latency benchmarks demonstrated sub-15ms response times for all validation endpoints. Because validation chains execute sequentially in-memory before database persistence, invalid payloads are rejected early, drastically conserving backend CPU and database connection resources.

11. COMPARATIVE ANALYSIS

To evaluate architectural trade-offs, the application design was compared across validation tiers and backend frameworks:

Evaluation Parameter	Client-Side Validation (ReactJS)	Native Node.js (http module)	ExpressJS + express-validator
Security Level	Low: Easily bypassed via curl / devtools	High: Authoritative server enforcement	High: Enterprise-grade chain validation
User Experience	Instant: 0ms UI feedback	Network round-trip latency (~15ms)	Near-instant interactive feedback (~15ms)
Code Modularity	Component state hooks (useState)	Manual URL and stream chunk parsing	Modular routers, middleware & controllers
Error Standardization	Custom React state error objects	Manual JSON string formatting	Standardized validationResult(req) arrays

12. CONCLUSION

The JobApply project successfully demonstrates the design, implementation, and cloud deployment of a complete web application showcasing server-side validation using ExpressJS and express-validator. All objectives specified in Practical No. 5 were accomplished:

- Developed an individual ExpressJS web application with clear MVC separation across models, controllers, middleware, and routes.
- Implemented 11 server-side validation chains covering full names, duplicate email checks, phone numbers, dynamic age calculations, degree checks, and experience bounds.
- Engineered RESTful endpoints handling GET, POST, and DELETE requests with standard HTTP status codes (200 OK, 201 Created, 400 Bad Request, 404 Not Found).
- Provided structured, field-level error diagnostics and an interactive live HTTP Response Inspector.
- Verified all 14 integration test cases (T1 to T14) with 100% pass rate.
- Deployed the application to GitHub Pages under the Atharv102 account with automated version control.